

Lab 2.1 — MAESTRO Automático (Generador de Threat Models con ADK)

Qué es este lab

Prueba si un agente puede aplicar el proceso MAESTRO de 6 pasos (descomposición → amenazas por capa → cross-layer → risk assessment → mitigaciones → monitoreo) a la descripción en texto de un sistema agéntico, y devolver el resultado como un objeto estructurado (`MaestroThreatModel`, un modelo Pydantic con 7 capas, amenazas, amenazas cross-layer y un top-5 de riesgos) en vez de prosa libre. No es un ataque -es la herramienta de análisis previa a todos los ataques del curso, el mismo framework con el que se clasifican los labs 2.2, 8.x, etc.

Las piezas que corren

- **`MaestroThreatModel` (Pydantic)**: el contrato de salida -7 capas nombradas, lista de amenazas por capa, amenazas cross-layer, top-5 riesgos. Define qué cuenta como "un threat model completo", no solo "una respuesta que suena razonable".
- **`analyze_layer` / `identify_cross_layer_threats`**: tools deterministas (Python puro) que resuelven nombres canónicos de capa/combinaciones -el agente las llama para anclar su razonamiento, no delegan ningún juicio de seguridad al LLM.
- **`maestro_agent`**: el agente tal cual lo define el libro
 - `output_schema=MaestroThreatModel` + `tools=[...]`, modelo qwen3.5:9b local vía Ollama (`LiteLlm`, `num_ctx=16384`, `temperature=0.2`, `reasoning_effort="none"`).
- **`maestro_agent_manual`**: agente alternativo -mismas tools, mismo modelo, pero SIN `output_schema` nativo: el schema esperado se describe como texto plano en la instrucción, y el parseo es 100% manual del lado de Python.

El flujo

1. El driver arma un prompt con la descripción del sistema a analizar (3 sistemas de prueba: agente de análisis de código, RAG de soporte, agente de trading).
2. **Intento 1**: se consulta a `maestro_agent` (nativo). Durante el razonamiento llama `analyze_layer` una vez por cada una de las 7 capas y `identify_cross_layer_threats` para las combinaciones que encuentre.

3. El texto final se intenta parsear como `MaestroThreatModel` con una estrategia robusta de 2 pasos (parseo directo, y si falla, extracción del objeto JSON balanceado de en medio de prosa/fences antes de reintentar) -nunca con `model_validate_json` desnudo.
4. Si el intento 1 no produce un JSON válido para el schema, **intento 2**: se repite todo el proceso con `maestro_agent_manual` (sin `output_schema` nativo, schema en texto).
5. Si ambos intentos fallan, el lab no revienta: devuelve un resultado de fallo explícito con diagnóstico, en vez de una excepción no controlada.

Por qué importa la verificación

Cada tool (`analyze_layer`, `identify_cross_layer_threats`) imprime `[tool call ejecutado] ...` -evidencia de que el modelo realmente invocó el razonamiento por capas, no que "sonó como si lo hubiera hecho". Además, `analyze_layer` registra cada capa vista en un set en memoria, y `build_threat_model` imprime ese set después de cada intento (`Capas analizadas via tool: [1, 2, ...]`) -esto es lo que permite distinguir, en una corrida real, si el modelo pasó por las 7 capas antes de cerrar el JSON o si cortó camino directo a la respuesta final. Más importante todavía: el parseo final nunca asume que el texto del modelo es JSON válido -el resultado se valida en 3 niveles (parseo directo → extracción y reintento → fallback a un agente sin `output_schema` nativo) antes de darse por vencido, y cada nivel queda registrado explícitamente en la salida (qué intento funcionó, o por qué fallaron los dos).

Resultado esperado

Una corrida exitosa imprime, para cada uno de los 3 sistemas de prueba, qué modo se usó (`native` o `manual_fallback`), el threat model completo (capas, amenazas, top-5 riesgos, amenazas cross-layer) y evidencia de que se llamaron las 7 `analyze_layer` antes del reporte final. Una corrida degradada (si ambos intentos fallan para algún sistema) imprime `modo=failed` con el texto crudo de ambos intentos para diagnóstico -el proceso sigue vivo y sigue con el siguiente sistema de prueba.

Es el lab más pesado del curso en tiempo de cómputo: un intento nativo puede tardar varios minutos, y el fallback manual (cuando hace falta) bastante más, porque ahí sí se ejecutan de verdad las 7 llamadas a `analyze_layer` más varias de `identify_cross_layer_threats`. Con los 3 sistemas de prueba, si alguno necesita ambos intentos, el proceso completo puede rondar los 50 minutos. Para no gastar tiempo de más, conviene correr un sistema por vez en vez de los 3 juntos.